

Traiter les questions suivantes:

I- Définir et expliquer :

- Intégrité des données
- Efficacité du logiciel

II- Expliquer la différence entre un test en boîte blanche et un test en boîte noire.

III- Parler de l'architecture centrée sur l'événement. Donner un exemple justificatif.

IV- Décrire les différences entre la vérification et la validation, et expliquer pourquoi la validation est particulièrement un processus difficile.

V-

- a) Qu'est ce qu'une :
 - Couverture d'instructions?
 - Couverture des arcs?
 - Couverture des chemins?
- b) Qu'est ce qu'un chemin indépendant ?
- c) Que signifie une région dans un graphe de contrôle ?

VI-Trouver le nombre cyclomatique du graphe de contrôle associé au programme suivant et donner un ensemble de tests.

```
void tri_shell (tableau t ; int n) {  
    element inf= t [0];  
    int incr=n;  
    element L;  
    int i, k;  
    while (incr > 1) {  
        if (incr < 5)  
            incr=1;  
        else incr = (5 * incr-1)/11;  
        for ( i = inf; i < n ; i+incr) {  
            k = i - incr;  
            while( k >= inf) {  
                if (L < t [k]) {  
                    t [k+incr] = t [k];  
                    k= k - incr; }  
                else exit;  
            }  
            t [k+incr]=L;  
        }  
    }  
}
```

2 pts

3 pts

3 pts

4 pts

4 pts

4 pts

Treat the following questions:

I- Define and explain:

- Data integrity
- Software efficiency

II- Explain the difference between a white box test and a black box test.

III- Talk about the event based architecture. Give a justified example.

IV- Describe the differences between verification and validation, and explain why validation is particularly a difficult process.

V-

- a) What is a:
 - Instruction covering?
 - Edges covering?
 - Path covering?
- b) What is an independent path?
- c) What does a region in a control graph mean?

VI-Find the cyclomatic number of the control graph associated to the following program and give a set of tests.

```
void sort_shell (tableau t ; int n) {  
    element inf= t [0];  
    int incr=n;  
    element L;  
    int i, k;  
    while (incr > 1) {  
        if (incr < 5)  
            incr=1;  
        else incr = (5 * incr-1)/11;  
        for ( i = inf; i < n ; i+incr) {  
            k = i - incr;  
            while( k >= inf) {  
                if (L < t [k]) {  
                    t [k+incr] = t [k];  
                    k= k - incr; }  
                else exit;  
            }  
            t [k+incr]=L;  
        }  
    }  
}
```